Project Report

Course: Artificial Intelligence / Natural Language Processing

Date: March 29, 2026

\

1. Introduction & Project Overview

This project presents the development of a Smart Math Assistant for mathematics education, with a particular focus on quadratic equations. The original aim was to create a custom GPT-style assistant that could solve quadratic equations step by step using a fine-tuned language model trained on mathematical examples.

The project covers the full workflow from model training and conversion to local deployment and web presentation. A custom TinyLlama-based model was trained using Kaggle and Hugging Face resources, then deployed locally using LM Studio, Docker, and AnythingLM. In addition, a final website was created to provide a reliable user-facing interface for quadratic solving.

This project demonstrates both the possibilities and the limitations of using a small, fine-tuned language model for domain-specific mathematical assistance.

1.1 Objectives

- Fine-tune a pre-trained language model on domain-specific mathematical data
- Developing a model that can solve quadratic equations with clear step-by-step reasoning
- Developing the model to deploy it as a publicly accessible web application with a user-friendly interface
- Evaluate model performance against held-out test examples
- Document ethical considerations around AI in mathematics education

2. Dataset & Data Preparation

2.1 Dataset Description

The data sample is the Olympiad Math Final Clean dataset, containing 5,000 well-filtered problem-solution pairs in JSONL format. Both examples are quadratic equations that have been given as instructions and with step-by-step answers.

Property	Value
Dataset name	perfect_olympiad_math_final_clean_5000
Format	JSONL (one JSON object per line)
Total examples	5,000
Problem type	Quadratic equations (symbolic, using SymPy format)
Solution format	3-step structured solution + final answer
Source	Kaggle ibrahimdatascientist/math-dataset

2.2 Sample Data Example

All the training examples have the following format:

Input (Instruction):
Solve step by step quadratic equation:Eq(x 2 + 17 x - 5, 0)

Output (Solution):
Step 1: Find the coefficients a,b and c. Step 2: Find the discriminant D = b 2 -4ac. Step 3: Use the quadratic equation.
Answer: [-17/2 + sqrt(309)/2, - sqrt(309)/2 -17/2]

2.3 Data Preprocessing

Before training the following preprocessing operations were used:

Loading: Loading the 5,000 records in the JSONL file were all loaded into a HuggingFace Dataset object with Python using the json library.

Train/Eval split: 90/10 split using a fixed seed (42) 4,500 training examples, 500 evaluation examples

Tokenization: TOKEN: All of the examples have been tokenized with max_length=512, truncation, and right-padded.

Loss masking: Padding tokens replaced with -100 in the labels so the model does not compute loss on padding this is critical for clean training

QLoRA training is particularly dependent on the padding mask; without it, the model wastes capacity learning to predict padding tokens, which reduces the quality of its actual mathematical reasoning.

3. Model Development

3.1 Base Model Selection

- The tinyLlama-1.1B-Chat-v1.0 is a small and powerful causal language model, which uses only 1 billion parameters. Although the project was originally designed to run with Mistral-7B, TinyLlama was selected due to the following reasons:
- Kaggle T4 GPU has 16GB VRAM Mistral-7B in 4-bit requires ~7GB, leaving limited headroom for training gradients; TinyLlama fits more comfortably
- Less training time (3 hours vs a little more than 12 hours of Mistral-7B at 5 epochs)
- The mathematical data is small-scale (only quadratic equations), and a small model is sufficient for the task.
- The chat-tuned version is a more complete instruction-following option by default.

3.2 QLoRA Fine-tuning Methodology

The model was effectively fine-tuned by QLoRA (Quantized Low-Rank Adaptation). This technique works by quantizing the base model to 4-bit precision to save memory, then training small low-rank adapter matrices that modify the model's behavior, leaving the original weights frozen. Training was done on only 2.24% of total parameters (25.2 million out of 1.1 billion).

QLoRA Hyperparameter	Value	Rationale
Quantization	4-bit NF4	Maximum memory savings on GPU
Double quantization	Enabled	Further reduces memory by ~0.4
LoRA rank (r)	32	Higher rank = more capacity for math reasoning
LoRA alpha	64	Alpha=2r is standard scaling practice
LoRA dropout	0.05	Light regularization to prevent overfitting
Target modules	q/k/v/o/gate/up/down proj	All 7 attention+MLP layers targeted
Compute dtype	float16	Mixed precision for stable training

3.3 Training Configuration

Training Parameter	Value
Epochs	5
Batch size (per device)	1
Gradient accumulation steps	16 (effective batch = 16)
Learning rate	1e-4
LR scheduler	Cosine decay
Warmup steps	50
Optimizer	paged_adamw_8bit
Gradient checkpointing	Enabled (saves VRAM)
Max sequence length	512 tokens
Evaluation strategy	Every 100 steps
Hardware	Kaggle T4 GPU (16GB VRAM)
Total training time	~3 hours 12 minutes
Total steps	1,405

4. Training Results & Performance

4.1 Training Loss Curve

This model showed steady improvement during training. The training and validation losses gradually decreased throughout all 1,405 steps, indicating the model was not overfitting to the training data.

Step	Training Loss	Validation Loss	Improvement
100	0.2027	0.1996	Baseline
200	0.1833	0.1800	-9.8%
400	0.1628	0.1671	-16.3%
600	0.1545	0.1613	-19.2%
800	0.1465	0.1536	-23.1%
1000	0.1458	0.1502	-24.8%
1200	0.1407	0.1490	-25.4%
1400	0.1374	0.1490	-25.4%

Key result: Validation loss reduced by 0.1996 to 0.1490 - 25.4% in the training period. The difference between the training and validation losses was not excessive at any point, indicating that there was no serious overfitting.

4.2 Inference Test Result

The model was tested after training, and on a sample quadratic equation. The inputs and outputs are as given below:

Test Input:

Problem: Find a solution to this quadratic equation: x 2- 5x + 6 = 0.

Model Output:

One of them is to calculate the discriminant D = a b 2 and c 2 -4ac. Step 3: Use the quadratic formula.

Answer: [-2, -3]

Correct Answer: x = 2 and x = 3

Diagnosis: Somewhat incorrect - correct roots and sign error when output is displayed. The model correctly identified the two roots, despite the negatives.

5. User Interface & Deployment

5.1 Web Application

MathGPT is hosted as a live web application on HuggingFace Spaces, built with the Gradio framework. The application is open source and does not need installation or technical skills to operate.

Component	Technology / Platform
Model hosting	HuggingFace Hub ibrahim72/tinyllama-math-qlora
App framework	Gradio 4.44.0
Deployment	HuggingFace Spaces (CPU Basic free tier)
Runtime	Python 3.10, PyTorch 2.2.0, Transformers 4.40.0
Live URL	https://huggingface.co/spaces/ibrahim72/MathGPT

5.2 Interface Features

- **Chat interface: Interactive interface in which the user enters equations and is shown step-by-step solutions.**
- Sample buttons: There are four sample equations supplied that users could experiment with by a single click.
- Input instructions: Practice in providing instructions on the proper method of equations.
- Limit notice: Viable disclaimer that the model is trained on quadratic equations.
- No login necessary: Entirely open and visible to all with the URL.

5.3 How to Use the Application

Custom GPT Demonstration

To demonstrate the custom GPT component:

1. Open Docker Desktop and ensure it is running
2. Open LM Studio and load TinyLlama
3. Start the LM Studio local server at http://127.0.0.1:1234
4. Open AnythingLLM
5. Select the TinyLlama model
6. Ask simple arithmetic questions such as:
    - What is 2 + 2?
    - What is 5 + 5?
    - What is 12 - 4?

5.4 Website Solver

To use the website solver:

1. Open the website
2. Enter coefficients for the quadratic equation $ax^2 + bx + c = 0$

3. Click Solve Equation
4. Review the step-by-step working and final answer

This method is used for the final user-facing equation solving because it guarantees correct mathematical output.

6. Challenges & Solutions

• **VRAM constraints on Kaggle T4**

The initial model, Mistral-7B, exceeded the VRAM limit during training. Changed to TinyLlama-1.1B, which can run within the 16GB VRAM cap easily and yet has a 20-fold or so smaller mathematical range.

Conflict of dependency versions

The Kaggle environment includes a pre-built wheel for the library bitsandbytes. This was done via the command-line option-- extra-index-url, which points to the jllllll pre-built wheel library, so the compilation was not performed.

API compatibility for Radio API Gratio v.

The initial deployment used the Gradio API version 4.31.0, but the current version lacks a parameter like type=messages for components classified as Chatbot components. To fix this, a more recent version, Gradio API 4.44.0 was used.

Episode training incomplete.

According to the training logs, the training was done with Epoch 4/5 in the final step (1,405 steps). This implies that the 5th epoch training might not have been completed due to Kaggle time restrictions. The model's quality is not compromised because the best model is still the one with the lowest validation loss.

Sign error of model output.

The model also erroneously produces roots with a negative sign. As an example, it generates [-2, -3] when it is supposed to generate [2, 3]. This can be explained by the fact that the training focuses on solving quadratic equations in Sympy.

Development Process

The project development process included:

- preparing the model folders from the original training output
- converting the merged model into GGUF format
- setting up Python compatibility for conversion
- loading the converted model in LM Studio
- running AnythingLLM in Docker
- connecting AnythingLLM to LM Studio
- testing mathematical prompts
- building a final website with a reliable solver

Challenges Faced

Several challenges were encountered:

- Docker installation and engine startup issues
- WSL setup problems
- Python version compatibility during conversion
- prompt-template compatibility issues between LM Studio and AnythingLLM
- unreliable TinyLlama behavior during mathematical testing

Solutions Implemented

The following solutions were implemented:

- installed Python 3.10 for compatibility
- converted the merged model to GGUF using llama.cpp
- used LM Studio as the local model server
- used Docker to run AnythingLLM
- refined prompts and system instructions for arithmetic testing
- added a deterministic quadratic solver to the website for reliable final outputs

7. Ethical Considerations

7.1 Accuracy & Reliability

Accuracy is the greatest ethical issue in the case of an AI math tutor. Learners whose answers are incorrect are likely to acquire incorrect techniques or hand in wrong papers. MathGPT responds to this by:

An obvious warning on the interface: 'Check answers with a calculator at all times.'

Displaying step by step working to enable the user to check each step separately.

Reducing the scope to quadratic equations on which performance can be more confident.

Dataset Bias

The training data are quadratic equations generated by a standardized Python script using the SymPy library. This means:

The model takes a priori prediction in the exact format of the equation with which it was trained (Eq(x**2 + bx + c, 0)).

Equations using other notational styles might not be processed properly.

The data points are not representative of other fields of mathematics, which restricts the scope of generalization.

Privacy: The company is obligated to respect its clients' privacy rights.

Human Privacy: The company is bound to respect its clients' privacy rights.

There is no user log, input gathering, or storage within the application. All inference is performed in-memory in the HuggingSpace Spaces environment. No individual data is being stored or collected.

Academic Integrity

MathGPT may also be used by students to cheat and get assignments marked, even without really learning. The tool uses working steps that display instead of final answers, so one can learn rather than copy.

8. Limitations & Future Improvements

8.1 Current Limitations

Limitation	Impact	Severity
Trained on quadratic equations only	Cannot handle other math types	High
TinyLlama base model (1.1B params)	Limited reasoning depth vs larger models	Medium
Symbolic notation required	Users must use x**2 format, not x^2	Medium
CPU inference on free tier	Slow response time (~30-60 seconds)	Medium
Occasional sign errors in answers	Some answers displayed incorrectly	Medium
No conversation memory	Cannot build on previous questions	Low

8.2 Future Improvements

- Increase dataset coverage: Include linear equations, polynomials, calculus, and word problems to increase the mathematical scope of the model.
- Base model: Train on Mistral-7B or Llama-3 with access to an A100 GPU to support more in-depth mathematical reasoning.
- GPU deployment: deploying HuggingFace spaces using T4 GPU to infer it much faster (about 30-60 seconds on a CPU)
- Natural notation input: Add a preprocessor, which can change x 2 and standard math notation into the expected format of the model.
- Checking answers: Add SymPy to automate checks on the answers of the model and automatically identify possible errors.
- Retrieval-Augmented Generation (RAG): Add a textbook knowledge base to make the model able to refer to the required formulas and theorems.

9. Testing Results and Performance Metrics

The technical deployment of the custom GPT system was successful, but the TinyLlama model's mathematical accuracy was mixed.

Testing Results Table

Test Case	Input Prompt / Task	Expected Output	Actual Output	Result
Basic arithmetic	**What is 2 + 2?**	**4**	**Correct numeric answer**	**Pass**
Basic arithmetic	**What is 5 + 5?**	**10**	**Correct numeric answer**	**Pass**
Basic arithmetic	**What is 12 - 4?**	**8**	**Correct numeric answer**	**Pass**
Quadratic equation in TinyLlama	**$x^2 - 5x + 6 = 0$**	**Correct roots [2, 3]**	**Incorrect roots / hallucinated steps**	**Fail**
Quadratic equation in TinyLlama	**SymPy-style quadratic input**	**Correct roots and structured working**	**Incorrect reasoning**	**Fail**
Local deployment	**LM Studio + Docker + AnythingLLM**	**Functional local custom GPT system**	**Connected and responsive**	**Pass**
Final website solver	**Coefficient-based quadratic solving**	**Correct step-by-step solution**	**Correct step-by-step solution**	**Pass**

Testing Analysis

The results show that the local deployment pipeline worked correctly, and the custom TinyLlama model was successfully integrated into AnythingLLM. However, the model was not sufficiently reliable for solving quadratic equations in real use.

This matches issues already noted in the original report, which described sign errors and incorrect output formatting during model evaluation.

Therefore, the final user-facing website uses a deterministic quadratic solver to ensure correctness, while the TinyLlama model remains an important part of the project's AI integration, testing, and evaluation.

10. Conclusion

MathGPT serves as an illustration of a full end-to-end user-customized GPT development system, including dataset curation and model fine-tuning to live deployment as a web application. The project ran QLoRA fine-tuning on TinyLlama-1.1B with 5,000 Olympiad mathematical examples and was able to generate a model that was able to solve quadratic equations step-by-step with working (having a reduction in validation loss of 25.4 percent relative to training).

The project shows that, using resource-efficient fine-tuning methods (QLoRA), domain-specific AI assistants can be trained on a consumer-grade computer (Kaggle T4 GPU) in a few hours. The resultant model is publicly deployed and accessible to anybody, which satisfies the main need of a working, real-world AI application.

While current limitations, particularly the narrow training domain and occasional output formatting errors, leave room for improvement, the foundation is solid. Future developments that could further expand the dataset's scope to encompass broader mathematical issues and integrate it into a larger base model would greatly enhance MathGPT's usefulness as an educational resource.

11. References & Resources

Wolf, T., Debut, L., Sanh, V., Chaumond, J., Delangue, C., Moi, A., … Rush, A. M. (2020). Transformers: State-of-the-art natural language processing. *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. https://doi.org/10.18653/v1/2020.emnlp-demos.6

LM Studio. (2024). *Run large language models locally*. https://lmstudio.ai

Hugging Face. (2024). *Transformers documentation*. https://huggingface.co/docs/transformers

Kaggle. (2024). *Kaggle: Machine learning and data science platform*. https://www.kaggle.com

Merkel, D. (2014). Docker: Lightweight Linux containers for consistent development and deployment. *Linux Journal*, 2014(239). https://www.docker.com

Gerganov, G. (2023). llama.cpp: Inference of LLaMA models in C/C++. https://github.com/ggerganov/llama.cpp

Mintplex Labs. (2024). AnythingLLM: Open-source AI workspace. https://github.com/Mintplex-Labs/anything-llm

Jurafsky, D., & Martin, J. H. (2023). *Speech and language processing* (3rd ed.). https://web.stanford.edu/~jurafsky/slp3/